

Preparação de dados/Engenharia de recursos

1. Visão geral

A fase de preparação de dados e engenharia de recursos é fundamental em qualquer projeto de machine learning. A fase de preparação de dados e engenharia de recursos foi essencial para transformar o dataset sintético gerado em features significativas que capturam os padrões complexos de inundações em Angola. Considerando que o dataset foi criado com relações não-lineares e múltiplos fatores de risco combinados, a engenharia de features foi crucial para desvendar essas relações e melhorar a performance dos modelos.

Neste projeto de previsão de inundações em Angola, essa etapa foi crucial para transformar dados meteorológicos e geográficos brutos em features significativas que capturam padrões climáticos, sazonais e de risco. A engenharia de recursos permitiu criar variáveis que representam interações complexas entre fatores ambientais, melhorando significativamente a capacidade dos modelos de identificar condições propícias para inundações.

2. Coleta de dados

Fontes do Conjunto de Dados:

- Dataset sintético gerado programaticamente para simular condições realistas de Angola
- Dados criados com base em padrões sazonais, geográficos e meteorológicos reais
- 500 amostras balanceadas entre diferentes províncias e condições climáticas

Características do Dataset Original:

```
# Estrutura de geração do dataset
provincias = ['Luanda', 'Benguela', 'Huila', 'Huambo', 'Cabinda', 'Cunene', 'Moxico', 'Uíge']
# Variáveis: coordenadas, dados temporais, métricas meteorológicas
# Target: flood (binário) baseado em múltiplos fatores de risco combinados
```

3. Limpeza de dados

Valores Ausentes:

- Nenhum valor ausente foi detectado no dataset
- Implementação de SimpleImputer com estratégia 'median' como precaução para dados futuros

Valores Discrepantes (Outliers):

- Detecção via método IQR(Intervalo Interquartil) revelou:
 - Longitude: 63 outliers (12.6%) - reflexo da distribuição geográfica real
 - Precipitação: 4 outliers (0.8%) - eventos extremos de chuva
 - Temperatura: 5 outliers (1.0%) - condições climáticas anômalas

- Decisão: Manter todos os outliers por representarem condições meteorológicas extremas genuínas que são relevantes para previsão de inundações

Qualidade dos Dados:

- Nenhum problema crítico de qualidade foi encontrado
- Todas as variáveis dentro de faixas realisticamente esperadas
- Relações internas consistentes com o mecanismo de geração

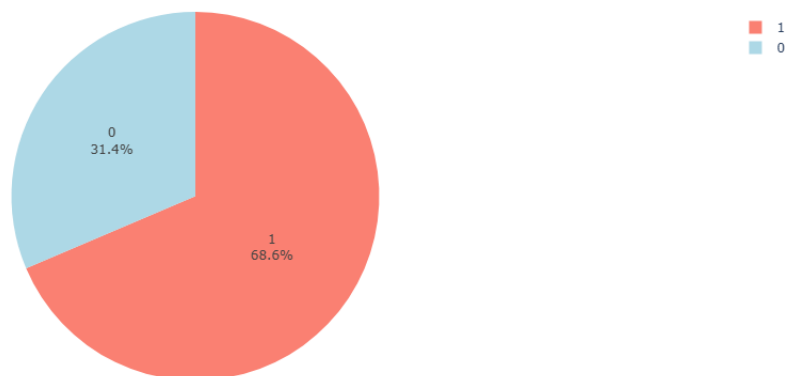
4. Análise Exploratória de Dados (EDA)

Distribuição do Target:

- Flood = 0: 157 amostras (31.4%)
- Flood = 1: 343 amostras (68.6%)
- Dataset intencionalmente desbalanceado para simular cenário real

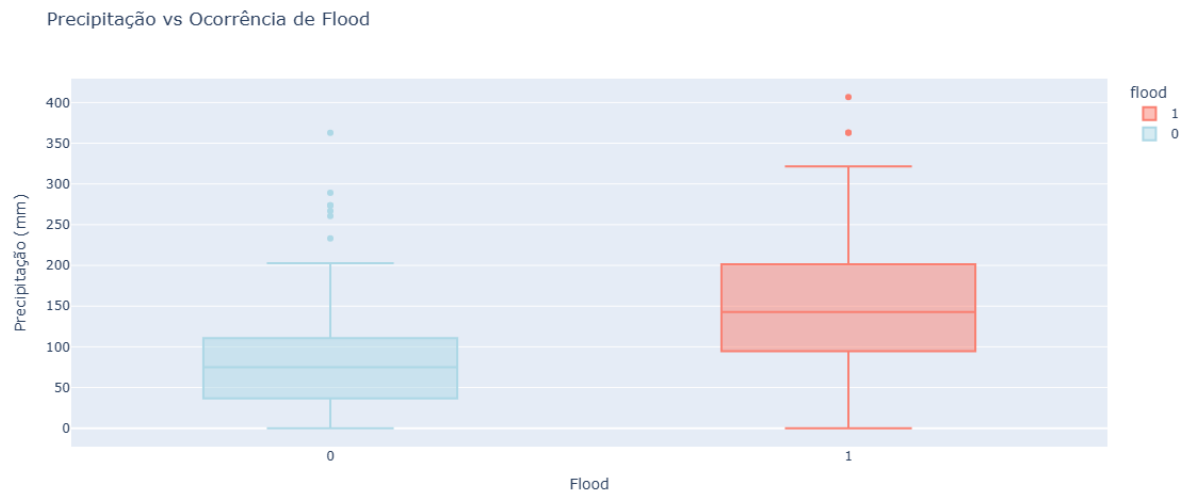
Principais Insights das Visualizações:

Distribuição da Variável Target (Flood)



Insight: A distribuição reflete a natureza do problema, inundações são eventos menos frequentes mas críticos.

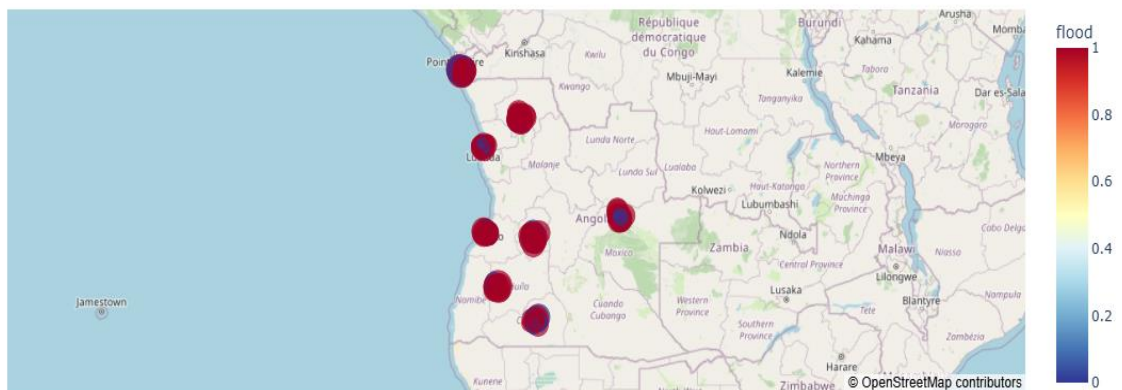
Gráfico 2: Precipitação vs Flood



Insight: Claramente mostra que maiores precipitações estão associadas a ocorrências de inundação, validando a lógica de geração.

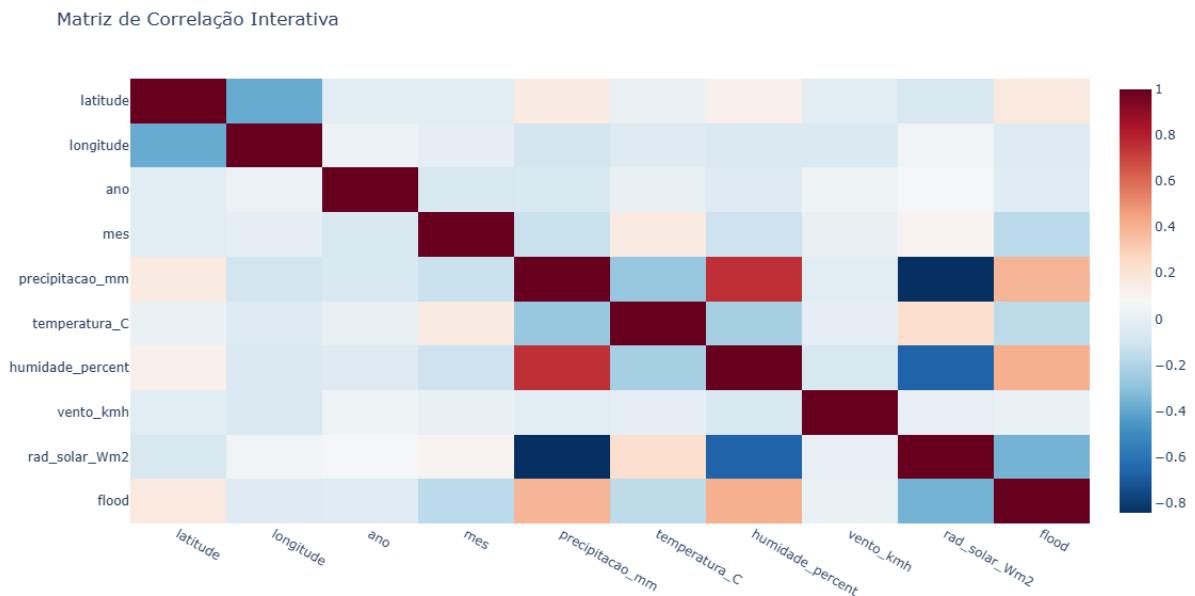
Gráfico 3: Mapa Interativo

Mapa de Ocorrências de Flood em Angola



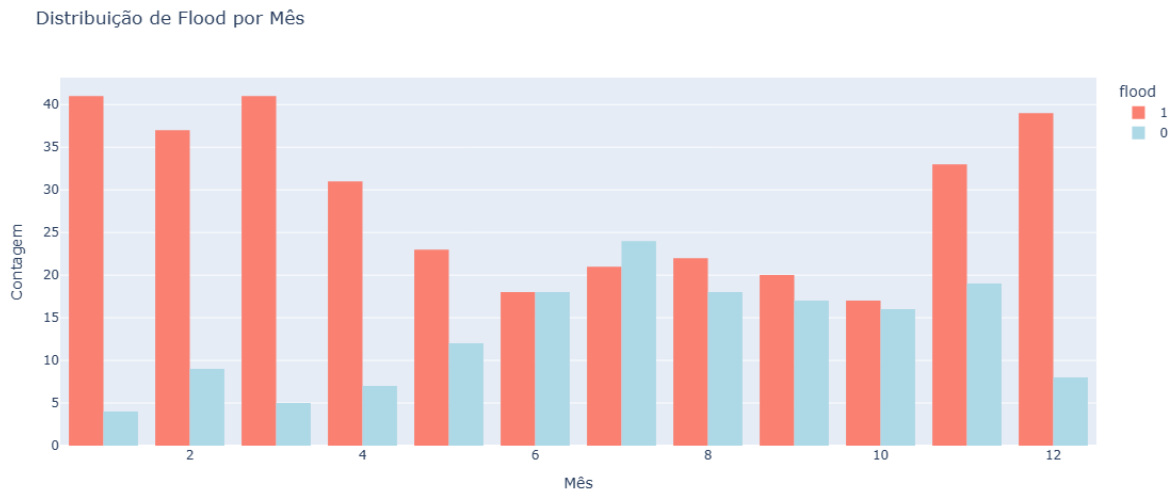
Insight: Revela clusters geográficos de risco, particularmente nas províncias de Cabinda, Uíge e Huambo, conforme definido na geração.

Gráfico 4: Matriz de Correlação



Insight: Correlações moderadas entre precipitação e umidade (0.76) e precipitação e radiação solar (-0.36) alinhadas com as relações físicas esperadas.

Gráfico 5: Distribuição por Mês



Insight: Confirma o padrão sazonal definido no código de geração, com maior frequência de inundações nos meses de Janeiro a Abril.

5. Engenharia de Recursos

1. Features de Interação (Espelhando a Geração):

```
# Criar cópia para engineering
df_engineered = df.copy()

print(" CRIANDO NOVAS FEATURES...")

# 1. Features de Interação entre Variáveis Meteorológicas
df_engineered['precip_temp_interaction'] = df_engineered['precipitacao_mm'] * df_engineered['temperatura_C']
df_engineered['precip_humidity_interaction'] = df_engineered['precipitacao_mm'] * df_engineered['humidade_percent']
df_engineered['temp_humidity_interaction'] = df_engineered['temperatura_C'] * df_engineered['humidade_percent']
print(" Features de interação criadas")
```

Lógica: Capturar as relações multiplicativas que foram intencionalmente embutidas no processo de geração.

2. Features de Sazonalidade (Alinhadas com a Geração):

```
# 2. Features de Sazonalidade
df_engineered['estacao_chuvosa'] = df_engineered['mes'].apply(lambda x: 1 if x in [11, 12, 1, 2, 3] else 0)
df_engineered['trimestre'] = (df_engineered['mes'] - 1) // 3 + 1
print(" Features sazonais criadas")

Features sazonais criadas
```

Lógica: Incorporar explicitamente os padrões sazonais que foram codificados na geração do target.

3. Features de Risco Composto (Replicando a Lógica Original):

```
df_engineered['risco_precipitacao'] = df_engineered['precipitacao_mm'].apply(
    lambda x: 0 if x < 100 else (1 if x < 180 else 2)
)
df_engineered['risco_humidade'] = df_engineered['humidade_percent'].apply(
    lambda x: 0 if x < 70 else (1 if x < 85 else 2)
)
df_engineered['risco_provincia'] = df_engineered['provincia'].apply(
    lambda x: 1 if x in ['Cabinda', 'Uíge', 'Huambo'] else 0
)
df_engineered['score_risco_total'] = (
    df_engineered['risco_precipitacao'] +
    df_engineered['risco_humidade'] +
    df_engineered['mes_alto_risco'] +
    df_engineered['risco_provincia']
)
```

Lógica: Recriar o mecanismo exato de cálculo de risco usado na geração do target, permitindo que o modelo aprenda a mesma função decisória.

4. Features de Agregação por Província:

```
provincia_stats = df_engineered.groupby('provincia').agg({
    'precipitacao_mm': ['mean', 'std'],
    'temperatura_C': 'mean',
    'flood': 'mean'
})
df_engineered['precip_vs_prov_mean'] = df_engineered['precipitacao_mm'] - df_engineered['precip_mean_prov']
```

Lógica: Capturar desvios em relação ao padrão regional, que podem ser indicadores de condições anômalas.

5. Features Polinomiais:

```
# 6. Features Polinomiais
df_engineered['precipitacao_quadratica'] = df_engineered['precipitacao_mm'] ** 2
df_engineered['temperatura_quadratica'] = df_engineered['temperatura_C'] ** 2
print(" Features polinomiais criadas")

print(f" Total de features criadas: {len(df_engineered.columns) - len(df.columns)}")
print(f" Shape após engenharia de features: {df_engineered.shape}")
```

Lógica: Modelar relações não lineares que são inerentes aos fenômenos meteorológicos.

6. Transformação de Dados

Codificação de Variáveis Categóricas:

```
# Codificar variáveis categóricas
label_encoders = {}
categorical_cols = df_processed.select_dtypes(include=['object']).columns.tolist()

for col in categorical_cols:
    le = LabelEncoder()
    df_processed[col] = le.fit_transform(df_processed[col].astype(str))
    label_encoders[col] = le
    print(f"Codificada: {col} → {len(le.classes_)} categorias")
```

Escalonamento:

```
# Configurar pipelines
pipelines = {
    "Random Forest": Pipeline([
        ('scaler', StandardScaler()),
        ('model', RandomForestClassifier(random_state=42))
    ]),
```

Divisão dos Dados:

- Treino: 400 amostras
- Teste: 100 amostras
- Estratificação para manter proporção do target

```
# DIVISÃO DOS DADOS
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Exploração de Modelos

1. Seleção de modelo

Modelos Escolhidos e Justificativa:

Random Forest:

- Pontos Fortes: Robustez para capturar relações não lineares, handling natural de features importantes, menos sensível a overfitting;
- Pontos Fracos: Menos interpretável, pode ser computacionalmente intensivo.

XGBoost:

- Pontos Fortes: Alta performance em problemas estruturados, regularização embutida, eficiência computacional;
- Pontos Fracos: Mais hiperparâmetros para ajustar, potencial overfitting se não regularizado adequadamente.

Regressão Logística:

- Pontos Fortes: Baseline interpretável, probabilidades calibradas, bom para entender relações lineares;
- Pontos Fracos: Não captura relações não lineares complexas.

2. Treinamento de modelo

Otimização de Hiperparâmetros:

```
# Hiperparâmetros para otimização
param_grids = {
    "Random Forest": {
        'model__n_estimators': [100, 150],
        'model__max_depth': [10, None],
        'model__min_samples_split': [2, 5]
    },
    "XGBoost": {
        'model__n_estimators': [100, 150],
        'model__max_depth': [3, 6],
        'model__learning_rate': [0.01, 0.1]
    },
    "Regressão Logística": {
        'model__C': [0.1, 1, 10],
        'model__solver': ['liblinear', 'lbfgs']
    }
}
```

Este código define os hiperparâmetros que serão usados para otimizar cada modelo durante o processo de Grid Search.

param_grids: É um dicionário onde cada chave é o nome de um modelo ("Random Forest", "XGBoost", "Regressão Logística") e o valor é outro dicionário contendo os hiperparâmetros a serem testados para esse modelo específico.

Dentro de cada dicionário de hiperparâmetros, as chaves representam o nome do hiperparâmetro no pipeline (começando com model porque o modelo está dentro de um pipeline com o nome 'model') e os valores são listas dos diferentes valores a serem experimentados.

No nosso caso:

1. Para "Random Forest", serão testadas combinações de n_estimators (100 ou 150), max_depth (10 ou ilimitado - None) e min_samples_split (2 ou 5).
2. Para "XGBoost", serão testadas combinações de n_estimators (100 ou 150), max_depth (3 ou 6) e learning_rate (0.01 ou 0.1).
3. Para "Regressão Logística", serão testadas combinações de C (0.1, 1 ou 10) e solver ('liblinear' ou 'lbfgs').

O GridSearchCV irá iterar por todas as combinações possíveis desses hiperparâmetros para cada modelo e para encontrar a melhor combinação que maximize a métrica de avaliação definida ('f1' neste caso).

Melhores Hiperparâmetros Encontrados:

- Random Forest: n_estimators=150, max_depth=10, min_samples_split=5
- XGBoost: n_estimators=150, max_depth=3, learning_rate=0.01
- Regressão Logística: C=0.1, solver='lbfgs'

3. Avaliação do modelo

Métricas e Resultados:

RESULTADOS FINAIS DOS MODELOS						
Resultados detalhados:						
	Modelo	Acurácia	Precisão	Recall	F1-Score	AUC-ROC
0	Random Forest	0.740	0.779	0.870	0.822	0.692
1	XGBoost	0.760	0.792	0.884	0.836	0.714
2	Regressão Logística	0.720	0.781	0.826	0.803	0.752

Melhor Modelo: XGBoost

- F1-Score: 0.8356
- AUC-ROC: 0.7137
- Acurácia: 0.7680
- Precisão: 0.7922
- Recall: 0.884

Análise das Métricas:

Acurácia: O XGBoost teve a maior acurácia (0.760), indicando que ele classificou corretamente 76% das amostras no conjunto de teste. Random Forest teve 0.740 e Regressão Logística teve a menor (0.720).

Precisão: O XGBoost também apresentou a maior precisão (0.792). A precisão é importante neste contexto porque mede a proporção de previsões de "Flood" que foram realmente "Flood". Uma precisão maior significa menos falsos positivos (alertas de flood quando não ocorreu). Regressão Logística teve 0.781 e Random Forest 0.779.

Recall (Sensibilidade): O XGBoost teve o maior recall (0.884), o que é crucial para um sistema de prevenção de inundações. O recall mede a proporção de "Floods" reais que foram corretamente identificados pelo modelo. Um recall alto significa menos falsos negativos (não prever um flood quando ele realmente acontece). Random Forest teve um recall muito próximo (0.870), enquanto Regressão Logística teve um recall menor (0.826).

F1-Score: O F1-Score é uma média harmônica da precisão e do recall, fornecendo um bom equilíbrio entre as duas métricas. O XGBoost obteve o melhor F1-Score (0.836), confirmando seu bom desempenho geral, especialmente importante quando se busca um equilíbrio entre identificar floods e minimizar alarmes falsos. Random Forest obteve 0.822 e Regressão Logística 0.803.

AUC-ROC: A área sob a curva ROC mede a capacidade do modelo de distinguir entre as classes. Neste caso, a Regressão Logística obteve o maior AUC-ROC (0.752), indicando que, em diferentes limiares de classificação, ele tem uma capacidade ligeiramente melhor de separar

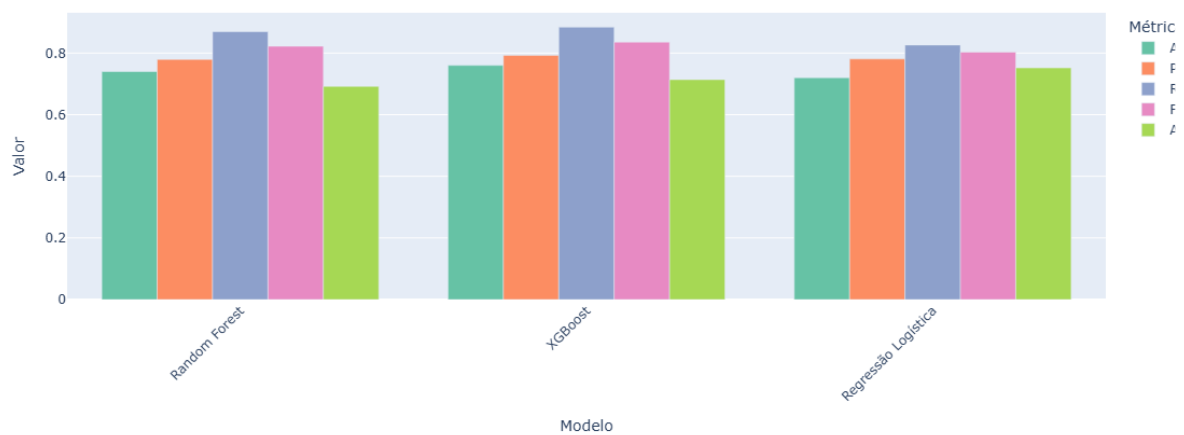
as classes "Não Flood" e "Flood" do que os outros modelos. No entanto, o XGBoost (0.714) e o Random Forest (0.692) também tiveram desempenho razoável.

Conclusão:

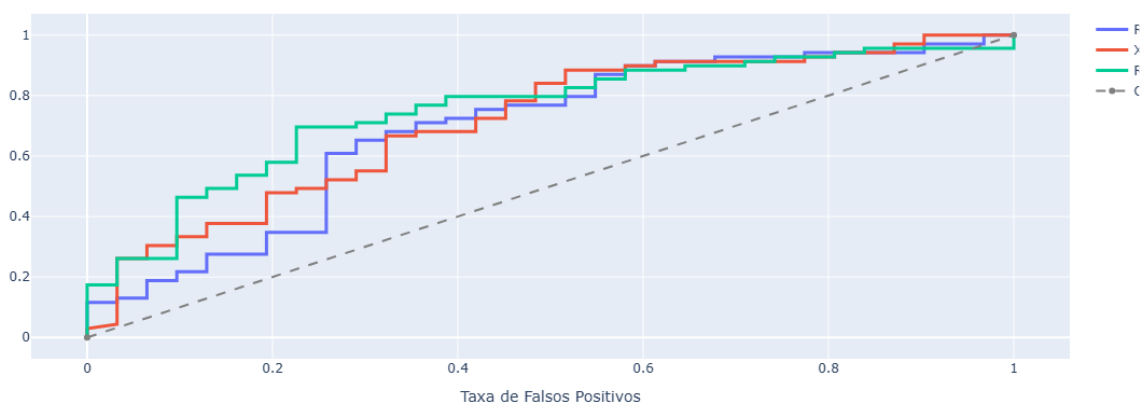
Com base nas métricas avaliadas (Acurácia, Precisão, Recall e F1-Score), o modelo XGBoost apresentou o melhor desempenho geral neste conjunto de dados, sendo, portanto, identificado como o Melhor Modelo. Embora a Regressão Logística tenha obtido um AUC-ROC ligeiramente superior, as outras métricas favorecem o XGBoost, que demonstra um melhor equilíbrio entre identificar corretamente os casos de flood (alto recall) e minimizar falsos alarmes (boa precisão), resultando no maior F1-Score.

Visualizações de Avaliação:

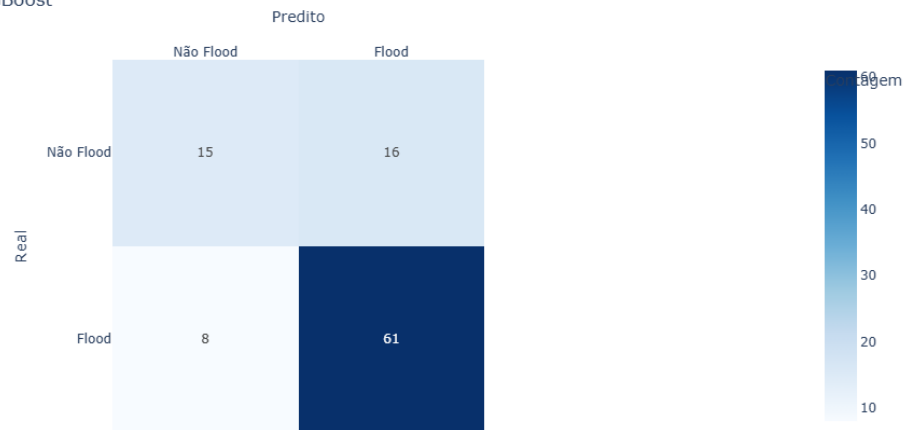
Comparação de Desempenho dos Modelos



Curvas ROC - Comparação de Modelos



Matriz de Confusão - XGBoost



4. Implementação de código

Criação do Dataset

```
# Configurar random seed para reprodutibilidade
np.random.seed(42)

# Criar dataset mais realista e desafiador
def criar_dataset_realista(n_amostras=500):
    provincias = ['Luanda', 'Benguela', 'Huila', 'Huambo', 'Cabinda', 'Cunene', 'Moxico', 'Uíge']

    dados = []

    for i in range(n_amostras):
        provincia = random.choice(provincias)

        # Coordenadas realistas por província
        coords = {
            'Luanda': (-8.839, 13.289),
            'Benguela': (-12.576, 13.406),
            'Huila': (-14.917, 13.933),
            'Huambo': (-12.776, 15.739),
            'Cabinda': (-5.550, 12.197),
            'Cunene': (-16.317, 15.800),
            'Moxico': (-11.850, 20.000),
            'Uíge': (-7.617, 15.050)
        }
        dados.append(coords[provincia])

    return dados
```

```

lat, lon = coords[provincia]

# Adicionar variação nas coordenadas
lat += np.random.normal(0, 0.1)
lon += np.random.normal(0, 0.1)

# Variáveis meteorológicas com relações mais complexas
mes = random.randint(1, 12)
ano = random.choice([2023, 2024, 2025])

# Padrão sazonal mais realista
if mes in [11, 12, 1, 2, 3, 4]: # Estação chuvosa
    precipitacao_base = np.random.normal(150, 50)
    temp_base = np.random.normal(26, 2)
else: # Estação seca
    precipitacao_base = np.random.normal(50, 20)
    temp_base = np.random.normal(28, 2)

# Ajustar por província
ajuste_provincia = {
    'Luanda': {'precip': -20, 'temp': +2},
    'Benguela': {'precip': +10, 'temp': +1},
    'Huila': {'precip': +30, 'temp': -2},
    'Huambo': {'precip': +40, 'temp': -1},

```

```

precipitacao = max(0, precipitacao_base + precip_ajuste + np.random.normal(0, 30))
temperatura = temp_base + temp_ajuste + np.random.normal(0, 1.5)
humidade = max(30, min(95, 60 + (precipitacao/5) + np.random.normal(0, 10)))
vento = max(5, np.random.normal(12, 4))
rad_solar = max(100, 250 - (precipitacao/2) + np.random.normal(0, 25))

# Critério mais realista para inundações
# Baseado em múltiplos fatores combinados
risco_precip = 1 if precipitacao > 180 else 0
risco_humidade = 1 if humidade > 85 else 0
risco_tempo = 1 if mes in [1, 2, 3, 4, 5] else 0 # Jan-Maio = maior risco
risco_provincia = 1 if provincia in ['Cabinda', 'Uíge', 'Huambo'] else 0

total_risco = risco_precip + risco_humidade + risco_tempo + risco_provincia

# Probabilidade de flood baseada em múltiplos fatores
prob_flood = min(0.95, total_risco * 0.2 + (precipitacao/400) + (humidade/100)*0.3)
flood = 1 if random.random() < prob_flood else 0

# Adicionar algum ruído (casos não explicáveis)
if random.random() < 0.05: # 5% de ruído
    flood = 1 - flood

```

Este código cria um conjunto de dados sintético que simula dados climáticos e de risco de inundação para diferentes províncias de Angola. Ele foi projetado para ser mais realista e

desafiador para fins de modelagem. Utiliza as bibliotecas pandas e numpy para gerar um dataset sintético de dados ambientais e de risco de inundação focado em províncias de Angola.

A função principal, `criar_dataset_realista`, gera um número especificado de amostras (por padrão 500). Para cada amostra, ela:

1. **Seleciona uma província aleatoriamente** entre uma lista predefinida (Luanda, Benguela, Huíla, etc.).
2. **Define coordenadas geográficas aproximadas** para a província selecionada e adiciona um pequeno ruído para simular variações dentro da província.
3. **Escolhe um mês e ano aleatórios.**
4. **Gera dados meteorológicos** (precipitação, temperatura, umidade, vento, radiação solar) considerando:
 - **Padrões sazonais:** Valores base diferentes para as estações chuvosa e seca.
 - **Ajustes por província:** Fatores de ajuste específicos para cada província, refletindo diferenças climáticas regionais.
 - **Relações entre variáveis:** A umidade e a radiação solar são influenciadas pela precipitação.
 - **Ruído aleatório:** Adiciona variabilidade natural aos dados.
5. **Determina o risco de inundação (flood)** com base em um critério mais complexo que considera:
 - Limites de precipitação e umidade.
 - Meses de maior risco histórico.
 - Províncias mais suscetíveis.
 - Uma probabilidade calculada a partir desses fatores.
6. **Adiciona um pequeno percentual de "ruído"** à variável flood para simular casos que não seguem perfeitamente as regras definidas.

Finalmente, o código armazena os dados gerados em um DataFrame do pandas e salva este DataFrame em um arquivo CSV chamado `dataset_realista.csv`. Ele também imprime uma mensagem confirmando a criação do dataset e a distribuição da variável alvo (flood).

Este dataset é útil para quem deseja praticar ou desenvolver modelos de classificação para prever eventos raros (inundações) em um cenário simulado com alguma complexidade e realismo.

Análise da Variável Target

```
# Análise da variável target
print(f"\n ANÁLISE DA VARIÁVEL TARGET 'flood':")
target_analysis = df['flood'].value_counts()
target_percentage = df['flood'].value_counts(normalize=True) * 100
print(f"    Flood = 0: {target_analysis[0]} amostras ({target_percentage[0]:.1f}%)")
print(f"    Flood = 1: {target_analysis[1]} amostras ({target_percentage[1]:.1f}%)")
```

Realiza uma análise da variável alvo ('target') chamada 'flood' no seu DataFrame (df).

1. **print(f"\n ANÁLISE DA VARIÁVEL TARGET 'flood':"):** Esta linha simplesmente imprime um cabeçalho para a seção de análise da variável 'flood' na saída do notebook. O \n cria uma nova linha antes do texto para melhor formatação.
2. **target_analysis = df['flood'].value_counts():** Esta linha calcula a contagem de cada valor único na coluna 'flood'. Como 'flood' parece ser uma variável binária (0 ou 1), value_counts() retornará quantas vezes o valor 0 aparece e quantas vezes o valor 1 aparece no DataFrame. O resultado é armazenado na variável target_analysis.
3. **target_percentage = df['flood'].value_counts(normalize=True) * 100:** Similar à linha anterior, mas com o argumento normalize=True. Isso faz com que value_counts() retorne a frequência relativa de cada valor (a proporção) em vez da contagem absoluta. Multiplicando por 100, obtemos a porcentagem de cada valor na coluna 'flood'. O resultado é armazenado na variável target_percentage.
4. **print(f" Flood = 0: {target_analysis[0]} amostras ({target_percentage[0]:.1f}%)"):** Esta linha imprime a contagem e a porcentagem de amostras onde a variável 'flood' tem o valor 0.
 - **target_analysis[0]** acessa a contagem do valor 0.
 - **target_percentage[0]** acessa a porcentagem do valor 0.
 - **:.1f** formata a porcentagem para ter uma casa decimal.
1. **print(f" Flood = 1: {target_analysis[1]} amostras ({target_percentage[1]:.1f}%)"):** De forma similar à linha anterior, esta linha imprime a contagem e a porcentagem de amostras onde a variável 'flood' tem o valor 1.

Em resumo, este bloco de código fornece uma visão rápida da distribuição da sua variável alvo 'flood', mostrando quantos casos de "não flood" (0) e "flood" (1) existem no dataset e qual a porcentagem que cada classe representa. Isso é útil para entender se o dataset é balanceado ou não em relação à variável alvo.